



Self-Driving Car Simulation Project Using CNN

Shu-Ting Hsu
Trinity Ma
Vladislav Zolotukhin

Table of Contents

Overview.....	2
Major Challenges & Solutions.....	2
I. Project architecture and modularity:.....	2
II. Environment setup and Dependency.....	3
Scenario 1: Anaconda prompt.....	3
Scenario 2: OpenCV.....	4
Scenario 3: Crashing script from AttributeError.....	4
III. Model Version Incompatibility.....	5
Instructions.....	5
I. Environment Setup and Dependencies.....	5
II. Creating a dataset.....	6
III. How To Launch Training/Testing.....	7
IV. Run the Simulation.....	10

Overview

Our approach to developing the autonomous driving model begins with gathering the dataset required using the Udacity Self-Driving Car Simulator, which captures images from center, left, and right camera perspectives along with their steering angles.

To efficiently train the model, we utilize a batch generator that applies a pipeline of preprocessing and data augmentation. The preprocessing stage prepares the raw images by isolating the road area, converting the color space to YUV, applying a 3x3 kernel Gaussian Blur to reduce noise, and resizing the frames to the default NVIDIA dimension of 200x66x3. To prevent overfitting and improve the model's generalization, random data augmentation techniques such as flipping, panning, zooming, rotating, and brightness adjustments were applied to a portion of the training data to increase dataset diversity, with the remaining for validation.

The predictive engine is a Convolutional Neural Network (CNN) model, progressing from normalized input planes through five convolutional feature maps (using 5x5 and 3x3 kernels) down to a series of fully-connected layers. A key difference is that our neural network's design uses the Exponential Linear Unit (ELU) activation function instead of the more common ReLU.

Although ELU is more computationally complex and results in longer initial training times, we end up selecting it because it successfully smooths at zero, mitigates the "dying neuron problem", and ultimately achieves a lower Mean Squared Error (MSE) with better convergence. The model iteratively trains on these augmented batches and evaluates its performance against an isolated validation dataset to compute the final MSE loss before exporting the resulting weights for the simulator.

Major Challenges & Solutions

There were a few obstacles that impacted the development of this project and they are as follows:

I. Project architecture and modularity:

When outlining the structure of the project, we followed the Machine Learning template and other fundamental programming architecture designs such as ensuring our code was modular and loosely coupled, each function has a specific

task and invokes other functions, and the code is designated to the appropriate module in the correct directory.

Another key reason why this decision was made is so that each project member could work on their specific tasks independently of others and avoid any merge conflicts or data duplication, seamlessly contributing to the project and reducing development time.

The outcome of this is that everything is organized into a coherent structure and decomposed into many smaller functions for specific purposes. The drawback to this was working with the correct relative paths and imports to ensure that all of the modules, functions, and files have clearly defined paths where they can be found that are independent of where the project is invoked from. Oftentimes, during the early stages of the project we encountered difficulty in unifying all the separate components of the project and specifying the correct paths when invoking functions.

This was also an issue for our visualizations, specifically the jupyter notebooks which ran scripts independently of the project entry point within their own confined code cells.

To mitigate this problem we implemented several different solutions, such as implementing "__init__.py" files to treat directories as packages, using a single entrypoint ([application.py](#)) into the project with support for different outcomes (train, test, or simulate), and using absolute paths. This was a significant early obstacle impacting the cohesion of our project and had to be resolved quickly to continue.

II. Environment setup and Dependency

Scenario 1: Anaconda prompt

The standard Anaconda/miniconda prompt somehow became completely unstable and would instantly crash upon opening. Later as we were debugging this issue, we noticed that the Windows environment variables were prioritizing a global Python 3.13 installation over the designated Conda environment, which created a pathing conflict where the system silently routed execution commands to the global directory instead of the local project space.



What we did to resolve this issue was to manually override it by using the absolute paths (e.g. `C:\...\miniconda3\envs\cvi620_env\python.exe`) directly to the environment's executable, successfully locking the execution state to the correct desired project boundaries.

Scenario 2: OpenCV

When attempting to import the libraries, it was stuck unable to import `cv2`, with the script kept throwing a `ModuleNotFoundError`. Multiple attempts to install the package resulted in '[WinError 5] Access is denied' message and crashes, with the module remaining undetected even after what was supposed to be successful installations.

With some research, we found two potential causes of this issue. First, the library's import name (`cv2`) differs from its actual package name (`opencv-python`), which may have caused the initial standard pip requests to fail. Second, the terminal used (Windows PowerShell) might have escaped the project's intended Conda environment for pip install and python commands, to using the global Python 3.13 installation due to system's background variables being forcefully routed by setting.

We bypassed the terminal's broken routing entirely by forcing the terminal to use absolute paths pointing directly to the environment's internal executables when installing. This guaranteed that OpenCV was installed strictly within the isolated Conda environment, making it instantly recognizable to the `predict_model.py` script.

Scenario 3: Crashing script from AttributeError

While attempting to run the simulation resulted in the scripts immediately crashing, throwing an `AttributeError` for module 'numpy' not having attribute 'object'. Turns out when installing Matplotlib or Pandas, pip silently upgraded Numpy to version 1.20+. Because pip default is to grab the newest sub-dependencies, yet for us we were bound to use TensorFlow 2.3.0 to maintain parity with the Udacity simulator, the `np.object` attribute that the older version of TensorFlow relied upon was deprecated with this replaced update, causing the script to crash.

Once we identified this was the issue, we audited the environment and manually downgraded and pinned every overlapping dependency to its exact 2020-era counterpart (e.g., NumPy 1.18.5, SciPy 1.4.1).

III. Model Version Incompatibility

While trying to run the scripts, it threw fatal `ValueError` and `TypeError` exceptions the moment they attempted to run the model. To figure out why it was crashing, we had to trace the errors back to the model's metadata. We discovered that the issue was with the architecture schema saved inside the file. This model was using a newer version of Keras/TensorFlow, which uses a newer term of `batch_shape` instead of the older `batch_input_shape`, and storing data types as `DTypePolicy` dictionaries rather than the expected text strings. Therefore, the local TensorFlow 2.3.0 environment could not understand the dialect.

Our first instinct was to upgrade our local environment to read the new file, but doing so severed the internal `tensorflow.keras` module bridge. To make sure that the code is consistent in the branch to prevent merge conflicts and break the project for other local environments.

Since we couldn't change the environment or the codebase, we decided to alter the file itself with a JSON manipulation script named [patch.py](#). What this script does is that it opens the localized `.h5` model file and extracts the hidden, byte-encoded `model_config` blueprint. It then acts as an automated translator to flatten the complex `DTypePolicy` objects back into simple strings, and `batch_shape` keyword back to `batch_input_shape`. With these changes, the older local environment was able to read and execute the `.h5` model that had newer dialects without altering the shared repository.

Instructions

The instructions for running this project can be viewed at any time from the [README.md](#) file at the root of the project. Note that the project structure follows the Machine Learning template outlined [here](#) and the project depends on the Udacity Self-Driving Simulator for both the dataset and the autonomous driving simulation, specifically the simulator executable file. This is a separate project and the release can be downloaded [here](#).

I. Environment Setup and Dependencies

Clone this repository to your local disk or download the zip directly from here and unpack to an empty directory.

From the project root run this line in your terminal (replace with the name of your virtual environment)

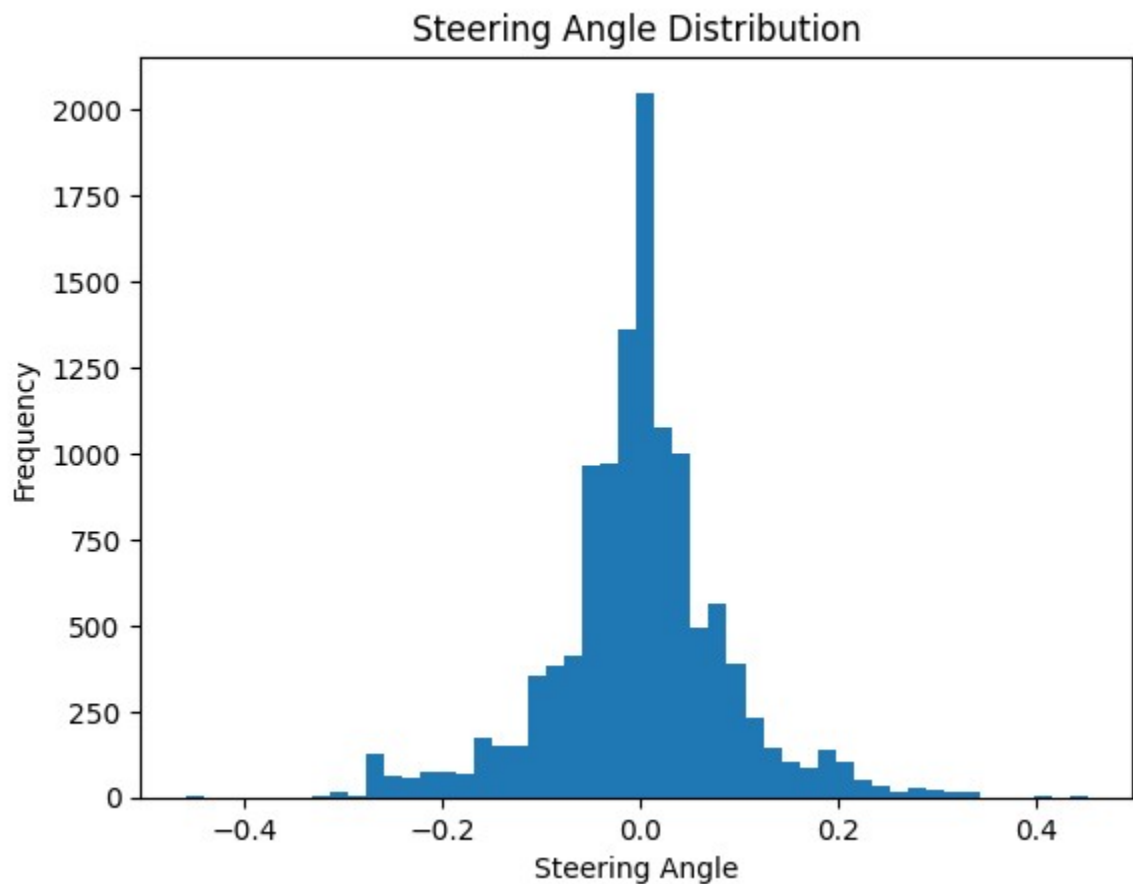
```
$ conda create --name <env> --file requirements.txt
```

Note: You need Miniconda installed to create a conda environment which can be downloaded [here](#). Alternatively, install the dependencies manually into your Python virtual environment. Please note that in order for the trained model to work in the simulator, it must run within the same virtual environment that the simulator was configured for. This means that some newer package versions may be incompatible and prevent the project server communicating with the simulator or otherwise result in malfunction. As a result, versions are explicitly defined in the package list (requirements.txt) and should be adhered to wherever possible.

II. Creating a dataset

Download the [Udacity Self-Driving Car Simulator](#). Launch the executable and enter training mode, record a training run of at least 5 full laps forward and another training run of 5 full laps driving in the opposite direction. Note that the model will be trained on this data and an insufficient sample size or incorrectly recorded data will result in discrepancies and poor performance. To increase performance it is suggested to use the mouse for driving and stick to the left lane of the track. This dataset should be saved into the /data directory of this project (e.g. /data/driving_log.csv, /data/IMG).

The compatibility of the dataset can be evaluated through the jupyter notebook "distribution.ipynb" which plots a histogram of the training data which should resemble a normal distribution such as this:



The histogram should avoid extreme outliers on left/right steering angle values. A balanced dataset indicates that the collected data has suitable distribution for training. Note that your distribution will look slightly different as it is dependent on your personal local dataset recorded by the driving simulator.

III. How To Launch Training/Testing

Run `application.py`

This will train and evaluate a model on the dataset, exporting the model into `src/model/versioning/model.h5`. If this file already exists, the model will only be evaluated instead.

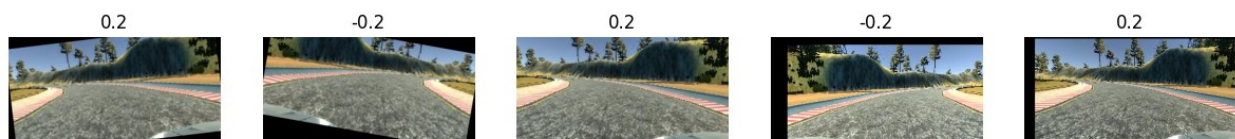
Please note that initial model training uses Elu activation function, using batches of preprocessed images.

Some research notes on Exponential Linear Unit (ELU) and Rectified Linear Unit (ReLU).

- They are both activation functions but differ in the following ways:
- Relu takes less computation as it invokes simpler operations making it faster
- However Relu is not as accurate as Elu due to the "dying neuron problem" and sharply smooths (V shape curve)
- Elu gradually smooths at 0 and mitigates the dead neurons, but is slower and more complex
- A Research study on 14,754 train images and 3,585 test images observed that ELU activation achieved lowest MSE compared to Relu and converged better.
- This model attempts to use elu to achieve better accuracy, but a consequence is that initial model training will take some time due to the complexity, number of epochs, and dataset size.
- Source paper can be read [here](#)

The train pipeline is as follows:

- Load the dataset (e.g. read the image paths and steering angles). Steering angles is a list of numeric values and the image paths are a list of lists of related images (left, center, and right - L/R have an offset). These lists must contain identical numbers of elements (for every steering angle, a list of 3 images)
- Generates batches from the training data that use image preprocessing and data augmentation in the batch generator. Preprocessing does the following:
 - Extracts only the road area from a source image, ignoring the top and bottom of the image
 - Convert to YUV color space
 - Resizes to the default NVIDIA size of 200x66x3 to be used for the simulator
 - Applies a Gaussian Blur using a 3x3 kernel to reduce noise
 - Normalizes the image
- Data Augmentation applies a random augmentation technique (flip, brightness adjust, zoom, rotate, pan, etc.) using random values (e.g. random rotation degree, random brightness adjust value, etc.) to an image to improve the generalization of the model by increasing the diversity of the training data. This is done randomly to a portion of the training data only. An example of this can be viewed in the jupyter notebook "transformation.ipynb" which applies this step to a random selection of 5 images from the dataset and plots them.



- Builds a CNN model (approach.py) using the outlined defined in Figure 7 of Final_Project.py:

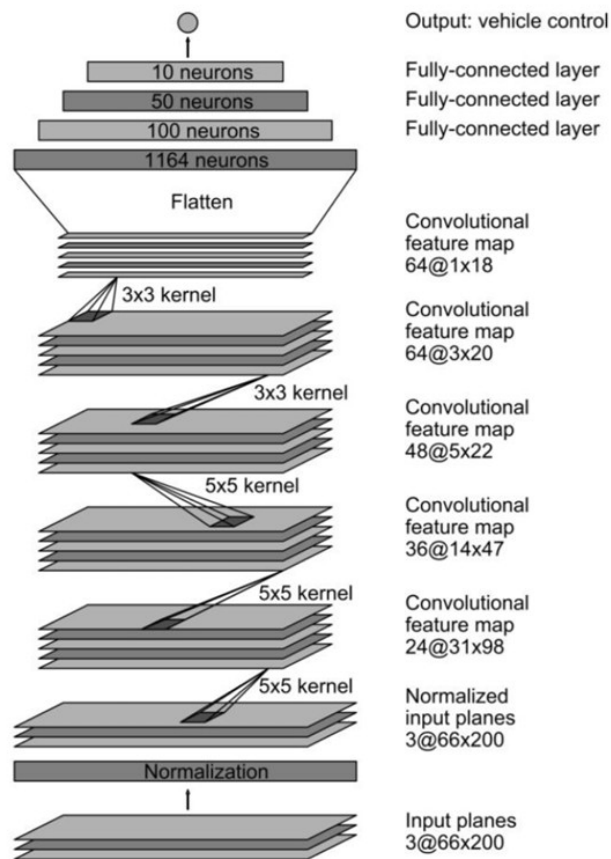
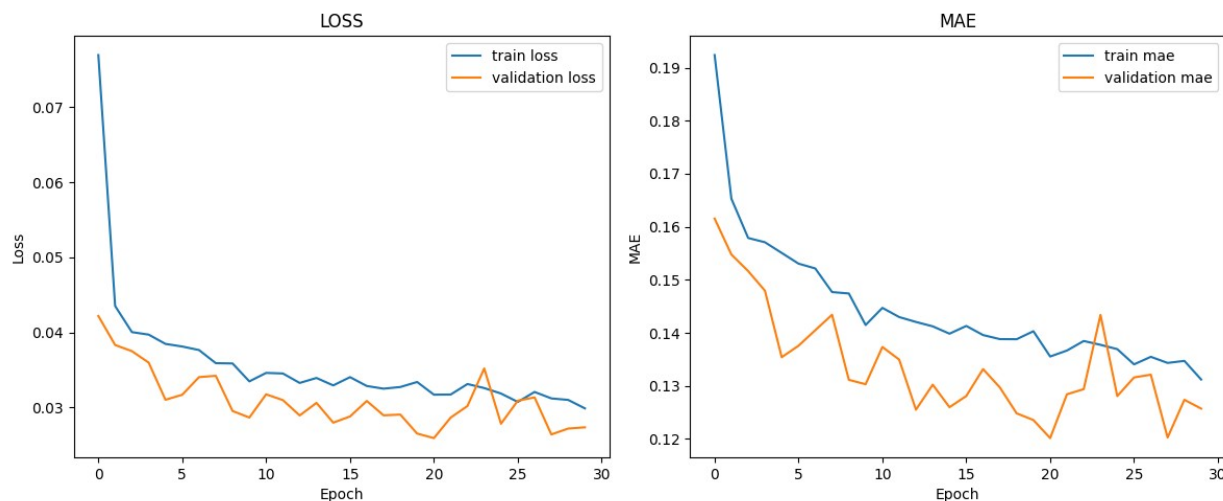


Figure 7.

- Exports the model to src/model/versioning/model.h5

After training, the training history is visualized using visualization_model.py. This module generates loss and MAE graphs for both training and validation datasets, allowing performance monitoring across epochs and helping identify convergence trends or possible overfitting.



The test pipeline is as follows:

- Evaluates the exported model in the specified path and evaluates it on the test data from the train/test split. If a model does not exist, it must first be trained and exported via the train pipeline, otherwise the existing model can be directly evaluated.
- Generates batches of the dataset to be evaluated upon, without applying any data augmentation and avoiding data leakage.
- Computes the final MSE loss value after evaluation.

IV. Run the Simulation

To run the simulation, run `application.py` with a "simulate" command-line argument:

```
$ python application.py simulate
```

The command-line argument 'simulate' specifies that the simulation script should be run (`run_simulate()`) instead of the default run function.

Open the Udacity Self-Driving Car Simulator executable, using the same settings configuration and select autonomous mode to autonomously drive the car using the model.